SYSTEM ARCHITECTURE SPECIFICATION v1.0

STATUS:
PRODUCTION-READY
BLUEPRINT

FEEDBACK-HUB

Multi-Tenant Client Observability, Triage & Real-Time Incident Bridge

Target Stack: PERN (PostgreSQL 16 + Express/Node 20 + React 18 + TS) | **Security:** Bank-Grade AES-256 + RLS + JWT Rotation | **Design:** Stripe Squarish System

1. EXECUTIVE IDEA EVALUATION & COMPETITIVE STRATEGY

Founder / Architect Honest Assessment:
Aapka idea **100% solid aur high-value enterprise problem** ko solve karta hai. Almost 85% tech companies mei sabse bada bottleneck yahi hota hai: *Client / Non-technical users bug face karte hain, WhatsApp ya email par vague screenshots bhejte hain ('bhai button kaam nahi kar raha'), aur internal API/Frontend dev team ke paas zero context hota hai (no headers, no console logs, no payload, no environment state).* Dev bolta hai 'Works on my machine', aur client frustrated hota hai.

STRATEGIC PILLAR	WHY THIS WINS (YOUR EDGE OVER JIRA / SENTRY)
Zero-Effort Telemetry	Client widget just 1-click submit karta hai. Background script auto-captures URL, User Agent, Network Failed Requests, Local Storage keys, Screen Res, and Redacted JWT context. Client ko form fill karne mei 5 seconds lagte hain.
Direct API Squad Dispatch	Routing rules ke through URL/API route ke hisab se specific squad (e.g. Auth team, Payment team) ke bucket mei directly ticket drop hoti hai with immediate Slack/WebSocket ding.
Dual-View Quarantine	Client portal ko sirf curated updates milte hain ('Investigating', 'Hotfix in QA', 'Resolved'). Internal dev discussions, stack traces, and dirty logs client se 100% hidden rehte hain.
Commercial B2B Potential	Pehle apni in-house products ke liye use karo. Once battle-tested, sell it as an Enterprise B2B SaaS competing with Usersnap / Marker.io par targeted at API-heavy Dev Shops.

2. DAY-1 MULTI-TENANT DATA ISOLATION (POSTGRESQL RLS)

Enterprise bank-level architecture requires guaranteed tenant isolation. Instead of costly database-per-tenant, we implement **Shared-Process, Shared-Database with PostgreSQL Row-Level Security (RLS)**. Har table me immutable tenant_id UUID hoga aur Postgres level par filter enforce hoga taaki ek tenant ka data kisi doosre tenant ko leak hone ka 0.00% chance rahe.

```
-- 1. Enable RLS on core tables ALTER TABLE tickets ENABLE ROW LEVEL SECURITY; ALTER TABLE ticket_events ENABLE ROW LEVEL SECURITY; -- 2. Define Tenant Isolation Policy via Session Variable CREATE POLICY tenant_isolation_policy ON tickets FOR ALL USING (tenant_id = NULLIF(current_setting('app.current_tenant_id', true), '')::UUID); -- 3. In Express.js Database Pool / Transaction Middleware: -- await client.query(`SET LOCAL app.current_tenant_id = $1`, [req.user.tenant_id]);
```

3. ENTERPRISE ROLE-BASED ACCESS CONTROL (RBAC) MATRIX

ROLE	SCOPE & PERMISSIONS	VISIBLE SURFACES	SECURITY CONTROLS

`super_admin`	Full platform control, tenant provisioning, global audit logs, billing override.	Global Root Portal (`/admin/*`)	Hardware MFA (FIDO2) + IP Whitelist
`tenant_admin`	Manage tenant users, invite devs/clients, create products, configure webhooks & SLA.	Tenant Settings, Billing, Squad Dashboards	TOTP MFA required + Session revocation
`squad_lead`	Assign tickets, override triage, merge duplicates, link internal Jira/GitHub issue, export reports.	Triage Queue, Dev Console, Telemetry Logs	Standard JWT + Tenant Binding
`developer`	View technical payloads, stack traces, network replays; write internal dev notes; mark resolved.	Full Tech Ticket View, Dev Notes, API Inspector	Client PII auto-redacted in logs
`support_agent`	Read sanitized ticket, post public customer messages, trigger manual escalation to dev squad.	Support Inbox, Client Messenger View	No access to raw DB/auth dumps
`client_user`	Submit feedback/bugs from embedded widget or client portal; track progress of their org's issues.	Client View Portal, Own Widget History	Strict zero-knowledge of internal dev notes
`widget_token`	Ingest-only public token embedded in web apps. Can only POST sanitized payload to /api/v1/ingest.	None (Headless SDK embed only)	Redis Token-Bucket Rate Limited + Domain Origin check

4. RELATIONAL DATA SCHEMA & AUDIT ARCHITECTURE

TABLE	KEY COLUMNS	DESCRIPTION & CONSTRAINTS
`tenants`	id (UUID PK), name, slug, plan, security_settings (JSONB), created_at	Multi-tenant organization boundary. Stores custom SLA thresholds and allowed domains.
`users`	id (UUID PK), tenant_id (FK), email, password_hash, role_id, mfa_secret, status	Argon2id password hashing, rotating refresh token family ID for instant compromise revoke.
`products`	id (UUID PK), tenant_id (FK), name, slug, api_key_hash, allowed_origins (ARRAY)	Represent distinct software products/services where the feedback widget is embedded.
`tickets`	id (UUID PK), tenant_id, product_id, title, description, category, severity, status, assigned_to	Indexed by `(tenant_id, status, severity, created_at)`. Auto-generated human short ID (e.g. FB-1042).
`ticket_telemetry`	ticket_id (FK), url, user_agent, network_logs (JSONB), console_errors (JSONB), screen_res, redacted_state	Sensitive PII is AES-256-GCM encrypted. Isolated from basic ticket table to keep queries ultra-fast.
`ticket_events`	id, ticket_id, author_id, event_type, content, is_internal_only (BOOLEAN), created_at	Event sourcing. `is_internal_only = true` is strictly filtered out for client roles.
`audit_logs`	id, tenant_id, actor_id, action, target_type, target_id, old_values, new_values, ip, user_agent	Append-only immutable audit trail required for SOC2 / ISO27001 bank-grade compliance.

5. STRIPE SQUARISH ENTERPRISE UI/UX SPECIFICATION

Aesthetic Mandate: High-contrast, sharp geometric structure reminiscent of Stripe Dashboard, Linear, and Vercel. Strict rounded-none or rounded-xs (2px) radius. No bubbly or generic templates.

UI ELEMENT	DESIGN SYSTEM TOKEN / RULE	INTERACTION / MICRO-ANIMATION
Corners & Borders	border-radius: 2px (Max 4px on modals). Crisp 1px solid #E4E4E7 (Light) / #27272A (Dark).	Zero pill shapes. Sharp industrial feel with precise 8px grid snapping.
Typography	Heading & Body: **Inter** / **Geist Sans** (tight tracking -0.02em). Code & IDs: **JetBrains Mono**.	Tabular numbers (font-variant-numeric: tabular-nums) for latency and error counts.

Color Palette	Background: #FFFFFF / #09090B. Surface Cards: #FAFAFA / #121214. Primary Accent: #4F46E5 (Indigo) or #000000.	Subtle contrast shift on hover (bg-neutral-50 to bg-neutral-100).
Command Menu (Cmd+K)	Global modal with instant search across all tickets, API squads, and error traces.	Sub-50ms instant filtering with keyboard arrow navigation.
Status Badges	Squarish tags with 1px border and pulsing status dot: [• OPEN], [• IN_DEV], [• RESOLVED].	Glowing subtle ambient dot for active real-time investigation.

6. EMBEDDABLE CLIENT SDK & INCIDENT DISPATCH ENGINE

End-to-End Pipeline:
1. **Client Product Embed:** Client adds <script src="https://hub.yourdomain.com/v1/sdk.js" data-product-key="prod_live_xxx"></script>.
2. **Auto-Telemetry Capture:** Intercepts window.onerror, unhandledrejection, and failed fetch calls (4xx/5xx). Sanitizes passwords/tokens via client-side regex.
3. **Fast Ingest Endpoint:** POST /api/v1/ingest/ticket validates API Key, applies Redis Token Bucket (max 30 req/min per IP to prevent spam), writes payload to PostgreSQL.
4. **Real-Time Event Dispatch:** Redis Pub/Sub broadcasts event to WebSocket gateway (Socket.io room tenant:{tenant_id}:squad:{squad_id}).
5. **Dev Instant Notification:** On-duty engineers get sound alert + visual badge in Feedback-Hub portal + Slack webhook alert within <150ms.

7. ZERO-DEFECT ENVIRONMENT ARCHITECTURE (.ENV)

ENV VARIABLE	SECURITY CLASSIFICATION	PURPOSE & ENFORCEMENT
`DATABASE_URL`	CRITICAL SECRET	PostgreSQL 16 connection with connection pooling (PgBouncer ready).
`REDIS_URL`	CONFIDENTIAL	Pub/Sub real-time gateway and Token-Bucket rate limiting store.
`JWT_ACCESS_SECRET`	HIGH SECRET (512-bit)	Short-lived 15-minute access token signing.
`JWT_REFRESH_SECRET`	HIGH SECRET (512-bit)	7-day rotating refresh token family secret.
`ENCRYPTION_KEY_AES256`	MASTER SECRET (32 bytes)	Used by backend to encrypt/decrypt sensitive PII and API keys in DB.
`CORS_ORIGIN_WHITELIST`	CONFIGURATION	Restricts API access to authorized frontend domains only.

READY FOR IMPLEMENTATION: Complete production codebase skeleton, SQL migration scripts, and frontend components have been configured in your workspace repository.